



SportDataPulse

Analyse des performances Ligue 1 - Saison 2025

Création, chargement et exploitation d'une base de données PostgreSQL



Modélisation



Requêtes SQL



Contrôles qualité

Business Intelligence Analyst

Steve Bell

Objectif et contexte

Transformer les données de Ligue 1 en aide à la décision sportive



551

joueurs analysés



18

clubs analysés



PostgreSQL

SGBD retenu



Valeur ajoutée BI

Passer d'une intuition sportive à une lecture factuelle, reproductible et défendable.



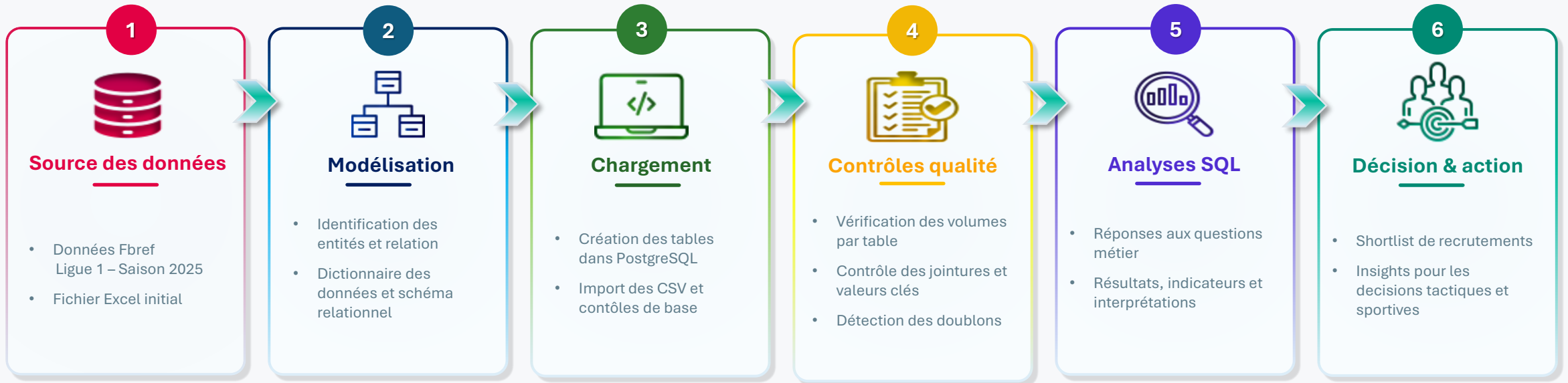
Mission

- Centraliser les statistiques des joueurs, des équipes, du temps de jeu, des xG, de la progression et des salaires.
- Répondre aux questions du directeur sportif avec des requêtes SQL vérifiables.
- Identifier les performances collectives, individuelles et les opportunités de recrutement.

- Décisions sportives objectivées
- Méthodologie SQL reproductible
- Contrôles qualité et traçabilité

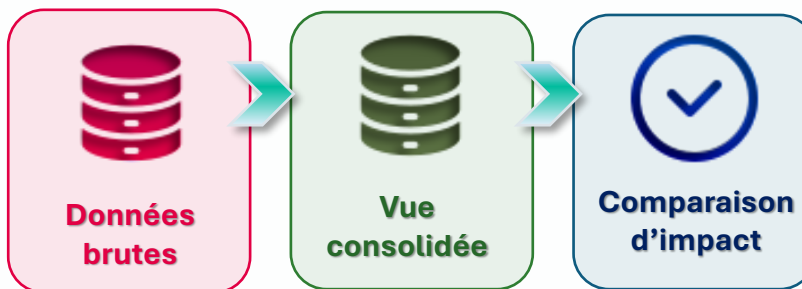
Cheminement du projet

De la donnée brute à la recommandation sportive



Principe directeur

Conserver les données brutes pour garantir la traçabilité, puis créer une vue consolidée afin de contrôler l'impact des doublons fonctionnels.



Objectif final

Fournir au staff sportif des analyses fiables, reproductibles et actionnables pour améliorer les performances et la stratégie du club

Données initiales et préparation

Source FBref : données joueurs, équipes et statistiques 2025



551

lignes joueurs



551

performances



18

équipes



Chaîne de traitement

1



Import du fichier source puis séparation en tables métier.

2



Contrôle des formats, valeurs manquantes et caractères spéciaux.

3



Extraction des CSV avec maintien de la cohérence des identifiants.

4



Détection de doublons fonctionnels et création d'une vue SQL consolidée.



Contrôles qualité ajoutés



Données brutes vs vue consolidée ;



Doublons fonctionnels ;



Écarts affichés uniquement si le résultat diffère.



Conclusion : l'impact global reste limité, mais la consolidation est pertinente pour fiabiliser les analyses par joueur.

Dictionnaire de données mis à jour

Structurer les champs pour fiabiliser les requêtes SQL



CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	RÈGLE DE GESTION	RÈGLE DE CALCUL
id_joueur	Identifiant unique du joueur issu des données source	Integer	NC	Élémentaire	Clé primaire ; non nul	Non calculé
nom_joueur	Nom et prénom du joueur	Varchar	100	Élémentaire	Non nul	Non calculé
nation	Nationalité du joueur	Varchar	50	Élémentaire	Peut être nul	Non calculé
annee_naissance	Année de naissance du joueur	Integer	4	Élémentaire	Peut être nul ; si renseignée, valeur cohérente	Non calculé
code_position	Position principale du joueur	Varchar	10	Élémentaire	Non nul ; clé étrangère vers position(code_position)	Non calculé
id_equipe	Identifiant de l'équipe du joueur	Integer	NC	Élémentaire	Non nul ; clé étrangère vers equipe(id_equipe)	Non calculé

Extrait du dictionnaire : codes, signification, types, longueurs



Structure documentée

- 8 tables documentées
- Champs renommés sans espaces ni accents
- Types SQL définis
- Clés primaires et étrangères précisées
- Règles de calcul indiquées



8 tables



PK / FK



Calculs



Pourquoi c'est important ?

- Normaliser les noms de champs.
- Définir les types SQL et contraintes.
- Faciliter les contrôles de cohérence.
- Rendre les requêtes compréhensibles et maintenables.



Traçabilité



Qualité



SQL

Schéma relationnel normalisé

Un modèle normalisé pour croiser les indicateurs sans dupliquer l'information

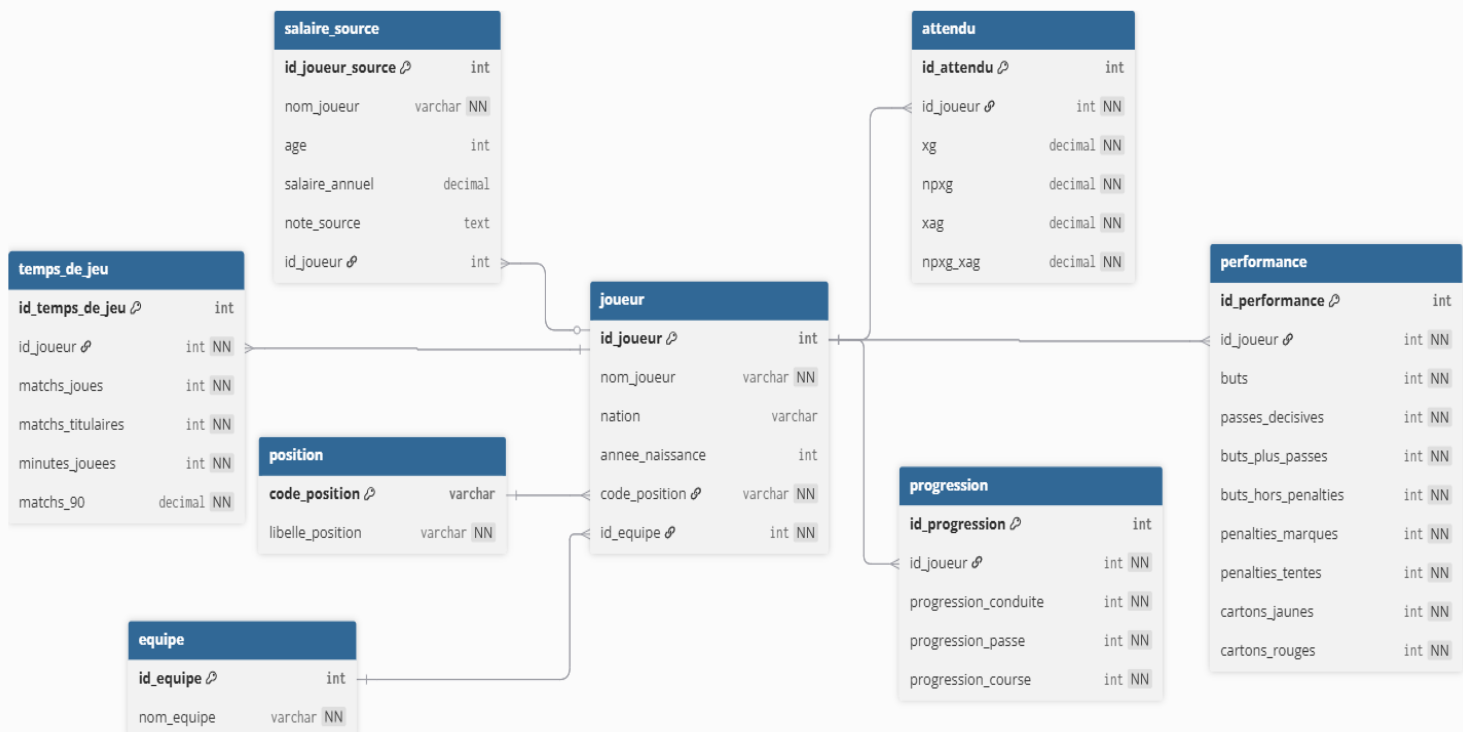


Schéma relationnel : joueur au centre, dimensions et tables statistiques



Choix de modélisation

- Table centrale : joueur.
- Référentiels : équipe, position.
- Statistiques séparées : temps de jeu, performance, xG, progression.
- Salaire conservé en source complémentaire.



Objectif : éviter la duplication de l'information, sécuriser les jointures et faciliter les analyses SQL.

Base PostgreSQL : tables chargées et vue consolidée

Contrôler la base sans modifier les données sources



pgAdmin / PostgreSQL

Schemas

public

Tables

- equipe
- joueur
- position
- performance
- temps_de_jeu
- attendu
- progression
- salaire_source

Views

- v_stats_joueurs_consolidees

Extrait simplifié de la vue consolidée

```
CREATE OR REPLACE VIEW v_stats_joueurs_consolidees AS
SELECT
  j.nom_joueur,
  j.code_position,
  j.id_equipe,
  SUM(p.buts) AS buts,
  SUM(t.matches_joues) AS matches_joues,
  SUM(a.xg) AS xg,
  SUM(pr.progression_passe) AS prgp
FROM joueur j
JOIN performance p ON j.id_joueur = p.id_joueur
JOIN temps_de_jeu t ON j.id_joueur = t.id_joueur
JOIN attendu a ON j.id_joueur = a.id_joueur
JOIN progression pr ON j.id_joueur = pr.id_joueur
GROUP BY j.nom_joueur, j.code_position, j.id_equipe;
```

Synthèse du chargement



8

tables métier



1

vue consolidée



0

modification des sources



SQL

reproductible



La vue consolidée permet de comparer les résultats bruts et consolidés, **sans modifier les tables sources.**

Contrôles de cohérence des données

Valider les volumes et la cohérence avant les analyses SQL



Exemple : contrôle des volumes après chargement

```
3 SELECT 'Joueurs' AS table_controlee, COUNT(*) AS total_lignes
4 FROM joueur
5
6 UNION ALL
7
8 SELECT 'Équipes' AS table_controlee, COUNT(*) AS total_lignes
9 FROM equipe
10
11 UNION ALL
12
13 SELECT 'Performances' AS table_controlee, COUNT(*) AS total_lignes
14 FROM performance
15
16 UNION ALL
17
18 SELECT 'Temps de jeu' AS table_controlee, COUNT(*) AS total_lignes
19 FROM temps_de_jeu
20
21 UNION ALL
22
23 SELECT 'Attendu' AS table_controlee, COUNT(*) AS total_lignes
24 FROM attendu
25
26 UNION ALL
27
28 SELECT 'Progression' AS table_controlee, COUNT(*) AS total_lignes
29 FROM progression
30
31 ORDER BY table_controlee;
```



Ces contrôles confirment que la base contient les joueurs, équipes et statistiques nécessaires avant de produire les indicateurs métier



Contrôles réalisés



Volumes par table : joueur, équipe, performance, temps de jeu, xG, progression.



Validité des jointures entre tables métier.



Division sécurisée avec NULLIF dans les ratios.



Comparaison brut vs consolidé pour mesurer l'impact des doublons.



Les contrôles limitent le risque d'analyse faussée par un mauvais import ou des valeurs incohérentes.



Résultat

	table_controlee ▼	total_lignes ▼
1	Attendu	551
2	Équipes	18
3	Joueurs	551
4	Performances	551
5	Progression	551
6	Temps de jeu	551

551

Joueurs
confirmés



Base prête pour
les analyses SQL

Données brutes vs vue consolidée

Mesurer l'impact des doublons fonctionnels sur les analyses



Approche 1 Données brutes

- Fidèles à la source.
- Alignées avec les résultats attendus.
- Conservent toutes les lignes historiques.

Utilisation :

Référence principale pour répondre aux attentes initiales et conserver une traçabilité complète.



Approche 2 Vue consolidée

- Fusion logique des doublons.
- Plus fiable pour les analyses par joueur.
- Impact global mesuré et documenté.

Utilisation :

Contrôle qualité permettant de fiabiliser les analyses centrées sur les joueurs.



Impact de la consolidation

source	nombre_lignes
joueurs bruts	551
joueurs consolidés	541

Exemple : Contrôle du nombre de lignes : brut vs consolidé

- **551 lignes brutes**
- **541 lignes consolidées**
- **-10 lignes après fusion logique**

La vue consolidée réduit les doublons fonctionnels sans modifier les tables sources. Les écarts restent limités, mais la démarche renforce la fiabilité des analyses par joueur.



Qualité data



Traçabilité

Requêtes SQL : méthode et exemple

Passer d'une question métier à un résultat SQL exploitable



Exemple de requête : total de buts par équipe

```
SELECT e.nom_equipe,  
       SUM(p.buts) AS total_buts  
FROM   equipe e  
JOIN   joueur j ON e.id_equipe = j.id_equipe  
JOIN   performance p ON j.id_joueur = p.id_joueur  
GROUP BY e.nom_equipe  
ORDER BY total_buts DESC;
```



Résultat obtenu

	nom_equipe ▼	total_buts ▼
1	Paris S-G	86
2	Marseille	71
3	Monaco	63
4	Lyon	62
5	Nice	56
6	Strasbourg	53
7	Brest	50
8	Lille	50



Structure documentaire systématique

Méthode utilisée pour chaque
besoin métier du projet.

1. Question métier

2. Requête SQL

3. Résultat

4. Interprétation

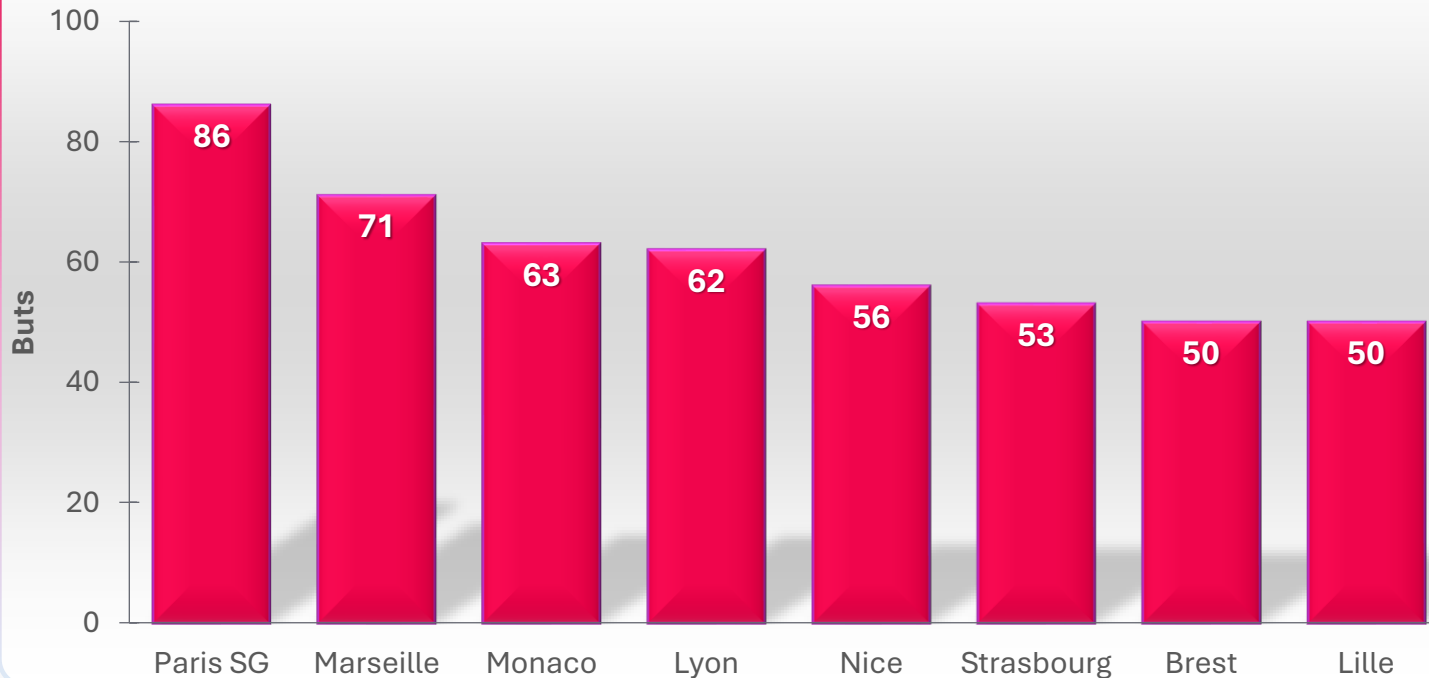
Résultat clé 1 : performance offensive des clubs

Comparer la production offensive des équipes avec le total de buts



 Requête 1

Total de buts par équipe



Lecture métier

Paris SG domine nettement la production offensive, devant Marseille, Monaco et Lyon.

- Indicateur collectif simple.
- Aide à comparer la puissance offensive.
- Base pour croiser avec les xG ou le temps de jeu.

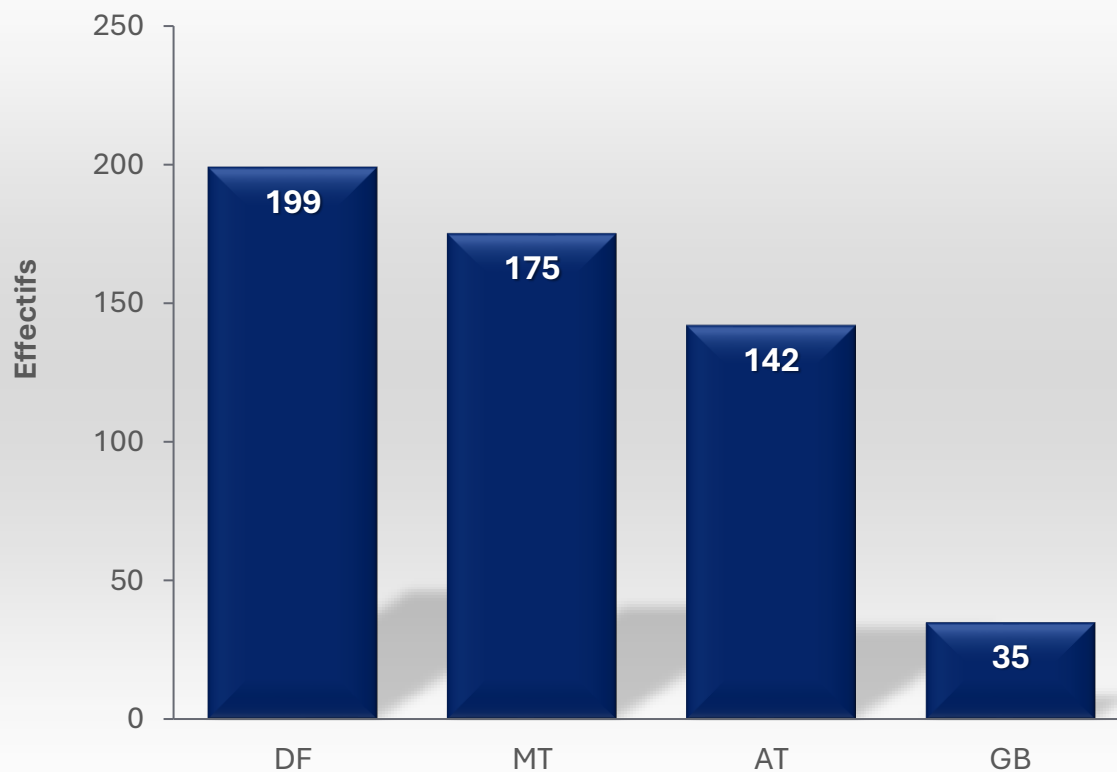
Résultat clé 2 : effectifs et répartition des postes

Analyser la structure des effectifs par type de poste



Requête 3

Répartition des joueurs par poste



DF : défenseurs · MT : milieux · AT : attaquants · GB : gardiens



Répartition observée

- Défenseurs et milieux représentent le cœur des effectifs.
- Les attaquants forment un volume important, mais plus ciblé.
- Les gardiens restent logiquement moins nombreux.



La répartition aide à contextualiser les moyennes par poste et à éviter les lectures biaisées.



Extrait SQL

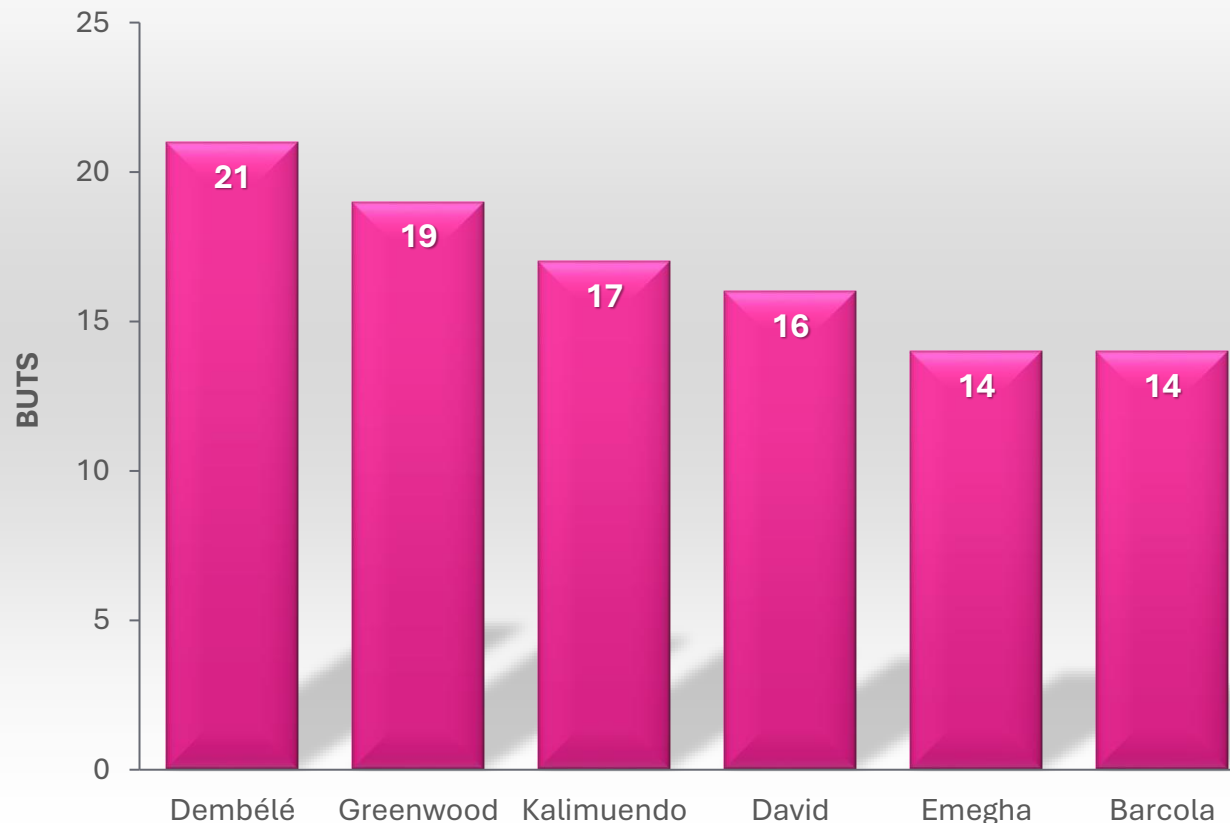
```
57 -- 3. Nombre de postes et % du total général pour chaque poste
58 SELECT
59     pos.code_position,
60     pos.libelle_position,
61     COUNT(j.id_joueur) AS nombre_joueurs,
62     ROUND(
63         COUNT(j.id_joueur) * 100.0
64         / SUM(COUNT(j.id_joueur)) OVER (),
65         2
66     ) AS pourcentage_total
67 FROM position pos
68     JOIN joueur j
69     ON pos.code_position = j.code_position
70 GROUP BY pos.code_position, pos.libelle_position
71 ORDER BY nombre_joueurs DESC;
```

Résultat clé 3 : efficacité individuelle

Identifier les finisseurs les plus performants au-delà du volume de buts



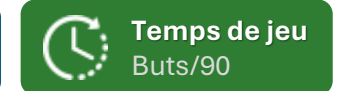
Top buteurs de Ligue 1



Au-delà du classement des buteurs

- Le top buteurs mesure le volume de buts.
- La différence buts - xG mesure l'efficacité de finition.
- Le ratio buts/90 compare les joueurs à temps de jeu équivalent.

La combinaison de ces métriques est plus robuste qu'un simple classement par buts.



Extrait SQL

```
99 SELECT
100     j.nom_joueur,
101     e.nom_equipe,
102     p.buts
103 FROM joueur j
104     JOIN equipe e 1..n<->1: ON j.id_equipe = e.id_equipe
105     JOIN performance p 1<->1..n: ON j.id_joueur = p.id_joueur
106 ORDER BY p.buts DESC
107 LIMIT 10;
```

Résultat clé 4 : discipline, temps de jeu et penalties

SportDataPulse · Ligue 1 2025 · Analyse BI



Discipline



Cartons rouges rapportés aux titularisations.

Repérer les profils à risque.



Requête 8

Temps de jeu



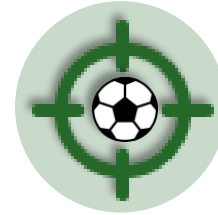
Minutes moyennes et matchs 90 par poste.

Comparer l'utilisation réelle des profils.



Requête 7

Penalties



Volume marqué + taux de réussite.

Identifier les spécialistes fiables.



Requête 5



Ces indicateurs replacent les chiffres dans leur contexte d'utilisation : un carton ou un but n'a pas le même poids selon le temps de jeu.

Analyse enrichie : meilleur joueur en PrgP par club

Identifier les moteurs de progression du ballon dans chaque club



Requête 10

```
153 SELECT
154     nom_equipe,
155     nom_joueur,
156     prgp
157 FROM (
158     SELECT
159         e.nom_equipe,
160         j.nom_joueur,
161         pr.progression_passe AS prgp,
162         ROW_NUMBER() OVER (
163             PARTITION BY e.id_equipe
164             ORDER BY pr.progression_passe DESC
165         ) AS rang
166     FROM joueur j
167     JOIN equipe e 1..n<->1: ON j.id_equipe = e.id_equipe
168     JOIN progression pr 1<->1..n: ON j.id_joueur = pr.id_joueur
169 ) classement
170 WHERE rang = 1
171 ORDER BY prgp DESC;
```

	nom_equipe	nom_joueur	prgp
1	Marseille	Pierre Højbjerg	258
2	Paris S-G	Vitinha	210
3	Lille	Benjamin André	199
4	Angers	Himad Abdelli	199
5	Lyon	Rayan Cherki	196
6	Montpellier	Téji Savanier	181

Extrait résultat : meilleur passeur progressif par club

Apport de la requête

```
ROW_NUMBER() OVER (
    PARTITION BY e.id_equipe
    ORDER BY pr.progression_passe DESC
) AS rang
WHERE rang = 1
```

- ➔ Garder uniquement le meilleur joueur de chaque club.
- ➔ Comparer les relais techniques par équipe.
- ➔ Repérer les joueurs moteurs dans la progression du ballon.



Lecture métier: la requête isole, pour chaque club, le joueur qui porte le plus la progression du ballon par la passe.

Recommandation recrutement : shortlist finisseurs



Identifier des attaquants accessibles avec un salaire annuel ≤ 3 M€



Critères SQL



Poste : **attaquant**



Salaire annuel ≤ 3 M€



Minimum **5 buts**



Classement prioritaire par nombre de buts ;



Ratio **PrgP / buts faible** pour cibler un profil **finisseur**.

Profil recherché : finisseur accessible



```
ROUND(pr.progression_passe::numeric /  
NULLIF(p.buts, 0), 2) AS ratio
```



Shortlist – 10 attaquants

#	Joueur	Club	Salaire	Buts	PrgP	Ratio Prgp/buts	
1	A. Kalimuendo	Rennes	1,5M€	17	71		4,18
2	J. David	Lille	1,95M€	16	63		3,94
3	E. Emegha	Strasbourg	0,75M€	14	14		1,00
4	M. Biereth	Monaco	1,15M€	13	24		1,85
5	L. Ajorque	Brest	2,16M€	13	41		3,15
6	E. Guessand	Nice	0,89M€	11	78		7,09
7	K. Nakamura	Reims	0,89M€	11	82		7,45
8	L. Stassin	Saint-Étienne	0,72M€	10	35		3,50
9	Z. Davitashvili	Saint-Étienne	0,9M€	9	68		7,56
10	E. Lepaul	Angers	0,27M€	9	17		1,89



Ratio PrgP/buts < 3



3 < Ratio PrgP/buts < 6



Ratio PrgP/buts > 6

Bonnes pratiques : sauvegarde, stockage et accès

Sécuriser la base, garantir la traçabilité et faciliter la maintenance



Sauvegarde

- Dump PostgreSQL planifié.
- Versions datées des sauvegardes.
- Test régulier de restauration.



Stockage

- Séparer les sources, la base, les scripts SQL et les exports d'analyse.



Accès

- Droits par rôle.
- Accès en lecture seule pour le staff non technique.



Traçabilité

- Conserver les scripts de création, de chargement, d'analyse et de comparaison.



Qualité

- Contrôles après import.
- Dictionnaire de données documenté et maintenu.



Objectif : rendre la base fiable, restaurable et exploitable par le staff dans la durée.

Annexe SQL 1/2 : requêtes métier et résultats

Synthèse des requêtes principales à commenter pendant la soutenance



But : montrer que chaque question métier est reliée à une requête SQL, un résultat et une interprétation. Les requêtes complètes restent dans le livrable Étape 4.

R2 - Moyenne de matchs joués par joueur, par équipe

```
SELECT e.nom_equipe,  
ROUND(AVG(t.matches_joues),2) AS moyenne  
FROM temps_de_jeu t  
JOIN joueur j ON t.id_joueur=j.id_joueur  
JOIN equipe e ON j.id_equipe=e.id_equipe  
GROUP BY e.nom_equipe;
```

Résultat : utilisation moyenne de l'effectif par club

Lecture : une moyenne élevée indique un effectif régulièrement utilisé ou une forte stabilité dans les choix de composition.

R5 - Top penalties + taux de réussite

```
SELECT j.nom_joueur, p.penalties_marques,  
p.penalties_tentes,  
ROUND(p.penalties_marques*100.0 /  
NULLIF(p.penalties_tentes,0),2) AS taux  
ORDER BY penalties_marques DESC LIMIT 10;
```

Résultat : spécialistes identifiés + taux sécurisé

Lecture : NULLIF évite la division par zéro ; le classement croise volume et fiabilité sur penalties.

R8 - Cartons rouges rapportés aux titularisations

```
SELECT j.nom_joueur, p.cartons_rouges,  
t.matches_titulaires,  
ROUND(p.cartons_rouges*100.0 /  
NULLIF(t.matches_titulaires,0),2) AS ratio  
WHERE p.cartons_rouges > 0;
```

Résultat : profils à risque relativisés par le temps de jeu

Lecture : un carton rouge est interprété selon le volume de titularisations, pas seulement en valeur brute.

R9 - Efficacité : différence entre buts et xG

```
SELECT j.nom_joueur, p.buts, a.xg,  
ROUND((p.buts::numeric - a.xg),2)  
AS difference_buts_xg  
FROM joueur j JOIN attendu a  
ON j.id_joueur=a.id_joueur  
ORDER BY difference_buts_xg DESC LIMIT 10;
```

Résultat : finisseurs au-dessus des occasions attendues

Lecture : l'écart buts - xG repère les joueurs qui convertissent mieux que la qualité moyenne de leurs occasions.

Annexe SQL 2/2 : analyses avancées et recrutement

Extraits SQL justifiant la valeur ajoutée BI et la shortlist



Ces requêtes répondent aux questions de recul : progression du ballon, efficacité par 90 minutes et recommandation de recrutement.

R10 - Meilleur PrgP par club

```
ROW_NUMBER() OVER (
  PARTITION BY e.id_equipe
  ORDER BY pr.progression_passe DESC
) AS rang
...
WHERE rang = 1;
```

Résultat : 1 joueur moteur de progression par équipe

Lecture : la fonction fenêtre permet de classer les joueurs à l'intérieur de chaque club et de conserver le meilleur relais progressif.

R11 - Shortlist attaquants finisseurs ≤ 3 M€

```
WITH candidats AS (...)
WHERE j.code_position LIKE '%AT%'
  AND s.salaire_annuel <= 3000000
  AND p.buts >= 5
ORDER BY buts DESC;
```

Résultat : 10 profils accessibles priorisés

Lecture : le filtre salaire + buts cible des profils exploitables ; le ratio PrgP/buts aide à repérer les finisseurs plutôt que les organisateurs.

R12 - Efficacité offensive : buts par 90 minutes

```
SELECT j.nom_joueur, p.buts, t.matches_90,
ROUND(p.buts::numeric /
NULLIF(t.matches_90,0),2) AS buts_par_90
WHERE t.matches_90 >= 5
ORDER BY buts_par_90 DESC LIMIT 10;
```

Résultat : comparaison neutralisée par le temps de jeu

Lecture : le seuil de 5 matchs complets évite de survaloriser un joueur ayant très peu joué.

R13 - Potentiel offensif avec temps de jeu limité

```
WHERE t.matches_90 BETWEEN 5 AND 20
  AND p.buts >= 3
ORDER BY buts_par_90 DESC,
xg_par_90 DESC, difference_buts_xg DESC
LIMIT 10;
```

Résultat : profils productifs malgré un rôle limité

Lecture : utile pour détecter des joueurs sous-exploités : efficacité réelle, qualité des occasions et minutes jouées.

Conclusion finale

SportDataPulse · Ligue 1 2025 · Analyse BI



La base transforme les données de Ligue 1 en décisions sportives plus objectives, argumentées et reproductibles.

1

Base fiable

PostgreSQL chargé, contrôlé et documenté.
Schéma relationnel, dictionnaire et vue consolidée assurent la traçabilité.

2

Analyses exploitables

Requêtes SQL reliées aux besoins métier : clubs, joueurs, postes, discipline, xG, PrgP.

3

Décision actionnable

Shortlist d'attaquants finisseurs ≤ 3 M€, à challenger ensuite par scouting vidéo et contexte contractuel.

Message final au staff

La donnée ne remplace pas le jugement sportif : elle le rend plus clair, plus comparable et mieux défendable.